Introdução à Linguagem Python

Visão Computacional em Robótica (BLU3040)

Prof. Marcos Matsuo (marcos.matsuo@ufsc.br)

Universidade Federal de Santa Catarina - UFSC

1. Liguagem Python: história e definições
2. Primeiros Passos com Python
3. Variáveis e Tipos de Dados
4. Operadores
5. Estruturas Condicionais
6. Estruturas de Repetição
7. Listas e Tuplas
8. Dicionários
9. Funções
10. Módulos e Pacotes

Liguagem Python: história e definições

- Criada por Guido van Rossum.
- Primeira versão lançada em 1991.
- Tornou-se uma das linguagens mais populares do mundo.
- Python é uma linguagem de programação de alto nível, interpretada e de propósito geral.
- Conhecida por sua sintaxe simples e legibilidade.
- Usada em diversas áreas como desenvolvimento web, ciência de dados, IA, automação, etc.

Olá, Mundo:

O primeiro programa em qualquer linguagem de programação geralmente é um simples "Olá, Mundo".

`arquivo exemplo_1.py`

```
1  # Código: Olá, Mundo
2  print("Olá, Mundo!")
```

Variáveis e Tipos de Dados

Variáveis são usadas para armazenar dados que podem ser utilizados e manipulados ao longo do programa.

Python é uma linguagem de **tipagem dinâmica**, o que significa que você não precisa declarar o tipo da variável explicitamente.

arquivo exemplo_2.py

```
1  # Inteiros
2  idade = 25
3  print("Idade:", idade)
4  print("Tipo de dado de 'idade':", type(idade))
5
6  # Flutuantes
7  altura = 1.75
8  print("Altura:", altura)
9  print("Tipo de dado de 'altura':", type(altura))
10
```

Variáveis e Tipos de Dados

Variáveis são usadas para armazenar dados que podem ser utilizados e manipulados ao longo do programa.

Python é uma linguagem de **tipagem dinâmica**, o que significa que você não precisa declarar o tipo da variável explicitamente.

arquivo exemplo_2.py

```
11  # Strings
12  nome = "João"
13  print("Nome:", nome)
14  print("Tipo de dado de 'nome':", type(nome))
15
16  # Booleanos
17  is_student = True
18  print("Estudante:", is_student)
19  print("Tipo de dado de 'is_student':", type(is_student))
```

Variáveis e Tipos de Dados

Embora Python seja uma linguagem de tipagem dinâmica, você pode usar **funções de conversão** para definir explicitamente o tipo de dado de uma variável no momento da sua criação.

arquivo exemplo_3.py

```
1  # Definindo tipos de dados explicitamente
2  idade = int(25)
3  altura = float(1.75)
4  nome = str("João")
5  is_student = bool(True)
6
7  print("Idade (com tipo explícito):", idade)
8  print("Tipo de dado de 'idade':", type(idade))
9
```

Variáveis e Tipos de Dados

Embora Python seja uma linguagem de tipagem dinâmica, você pode usar **funções de conversão** para definir explicitamente o tipo de dado de uma variável no momento da sua criação.

arquivo exemplo_3.py

```
10 print("Altura (com tipo explícito):", altura)
11 print("Tipo de dado de 'altura':", type(altura))
12
13 print("Nome (com tipo explícito):", nome)
14 print("Tipo de dado de 'nome':", type(nome))
15
16 print("Estudante (com tipo explícito):", is_student)
17 print("Tipo de dado de 'is_student':", type(is_student))
```

Operadores

Operadores são usados para realizar operações em variáveis e valores. Python suporta diversos tipos de operadores, incluindo aritméticos, relacionais, lógicos e de atribuição.

arquivo exemplo_4.py

```
1  # Operadores aritméticos
2  a = 10
3  b = 5
4
5  print("Soma:", a + b)          # Saída: 15
6  print("Subtração:", a - b)     # Saída: 5
7  print("Multiplicação:", a * b) # Saída: 50
8  print("Divisão:", a / b)       # Saída: 2.0
9  print("Módulo:", a % b)        # Saída: 0
10 print("Potência:", a ** b)     # Saída: 100000
```

Operadores

Operadores são usados para realizar operações em variáveis e valores. Python suporta diversos tipos de operadores, incluindo aritméticos, relacionais, lógicos e de atribuição.

arquivo exemplo_5.py

```
1  # Operadores relacionais
2  a = 10
3  b = 5
4
5  print("Igual a:", a == b)     # Saída: False
6  print("Diferente de:", a != b) # Saída: True
7  print("Maior que:", a > b)    # Saída: True
8  print("Menor que:", a < b)    # Saída: False
```

Estruturas Condicionais

Estruturas condicionais são usadas para tomar decisões no código com base em condições específicas. Em Python, usamos `if`, `elif` e `else` para implementar essas estruturas.

arquivo exemplo_6.py

```
1  idade = 18
2
3  if idade < 18:
4      print("Menor de idade")
5  elif idade == 18:
6      print("Acabou de atingir a maioridade")
7  else:
8      print("Maior de idade")
```

Estruturas de Repetição

Estruturas de repetição permitem executar um bloco de código várias vezes. Em Python, usamos `for` e `while` para implementar loops.

arquivo exemplo_7.py

```
1  # Percorrendo uma lista com for
2  frutas = ["maçã", "banana", "cereja"]
3
4  for fruta in frutas:
5      print(fruta)
```

Estruturas de Repetição

Estruturas de repetição permitem executar um bloco de código várias vezes. Em Python, usamos `for` e `while` para implementar loops.

arquivo exemplo_8.py

```
1  # Usando while
2  contador = 0
3
4  while contador < 5:
5      print("Contador:", contador)
6      contador += 1
```

Listas e Tuplas

Listas e tuplas são coleções de dados em Python. Listas são mutáveis (podem ser modificadas), enquanto tuplas são imutáveis (não podem ser modificadas após a criação).

arquivo exemplo_9.py

```
1  # Criando uma lista
2  numeros = [1, 2, 3, 4, 5]
3
4  # Acessando elementos
5  print("Primeiro elemento:", numeros[0])
6
7  # Adicionando um elemento
8  numeros.append(6)
9  print("Lista após adição:", numeros)
10
11 # Removendo um elemento
12 numeros.remove(3)
13 print("Lista após remoção:", numeros)
```

Listas e Tuplas

Listas e tuplas são coleções de dados em Python. Listas são mutáveis (podem ser modificadas), enquanto tuplas são imutáveis (não podem ser modificadas após a criação).

arquivo exemplo_10.py

```
1  # Criando uma tupla
2  cores = ("vermelho", "verde", "azul")
3
4  # Acessando elementos
5  print("Primeira cor:", cores[0])
6
7  # Tentando modificar uma tupla (gera erro)
8  # cores[0] = "amarelo"
```

Dicionários

Dicionários são coleções de pares chave-valor. Eles são mutáveis e permitem acesso rápido aos valores através das chaves.

arquivo exemplo_11.py

```
1  # Criando um dicionário
2  aluno = { "nome": "Maria", "idade": 22, "curso": "Engenharia"}
3
4  # Acessando valores
5  print("Nome:", aluno["nome"])
6
7  # Adicionando um novo par chave-valor
8  aluno["nota"] = 9.5
9  print("Dicionário atualizado:", aluno)
10
11 # Removendo um par chave-valor
12 del aluno["idade"]
13 print("Dicionário após remoção:", aluno)
```

Funções

Funções são blocos de código que podem ser reutilizados. Elas ajudam a modularizar o código e evitar repetição.

arquivo exemplo_12.py

```
1  def operacoes_basicas(a, b):
2      soma = a + b
3      subtracao = a - b
4      multiplicacao = a * b
5      # exemplo de uso de operador ternário (abaixo)
6      divisao = a / b if b != 0 else "Divisão por zero não é permitida"
7
8      return soma, subtracao, multiplicacao, divisao
9
10 # Exemplo de uso da função
11 a = 10
12 b = 5
13 r1, r2, r3, r4 = operacoes_basicas(a, b)
14
15 print(f"Soma: {r1}")
16 print(f"Subtração: {r2}")
17 print(f"Multiplicação: {r3}")
18 print(f"Divisão: {r4}")
```

Módulos e Pacotes

Módulos são arquivos que contêm definições e instruções Python.
Pacotes são coleções de módulos.

arquivo exemplo_13.py

```
1  # Importando um módulo inteiro
2  import math
3
4  # Usando uma função do módulo math
5  print("Raiz quadrada de 16:", math.sqrt(16))  # Saída: 4.0
6
7  # Importando apenas uma função específica
8  from math import pi
9
10 print("Valor de pi:", pi)  # Saída: 3.141592653589793
```